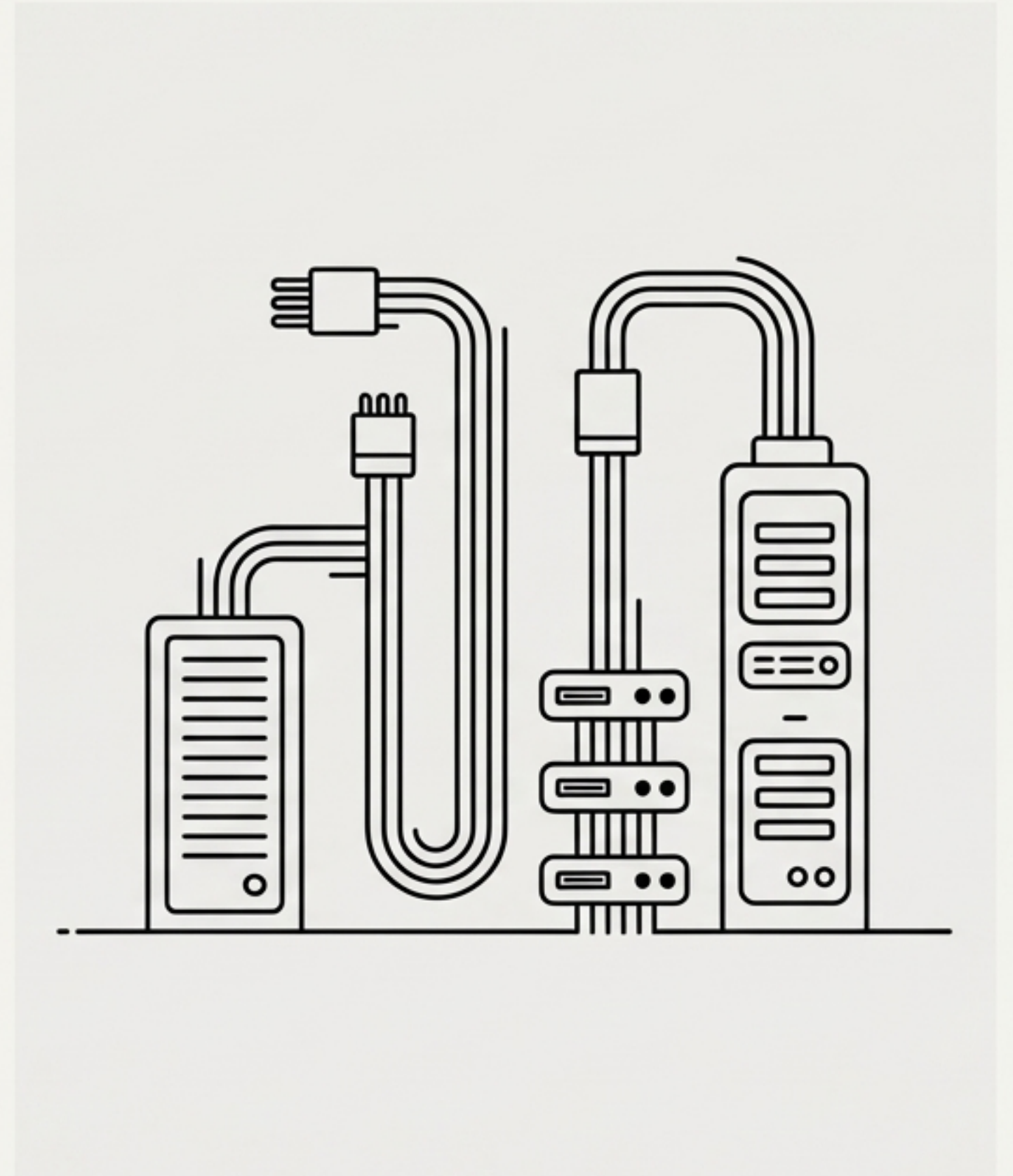


Redes no Docker: Comunicação entre Contêineres

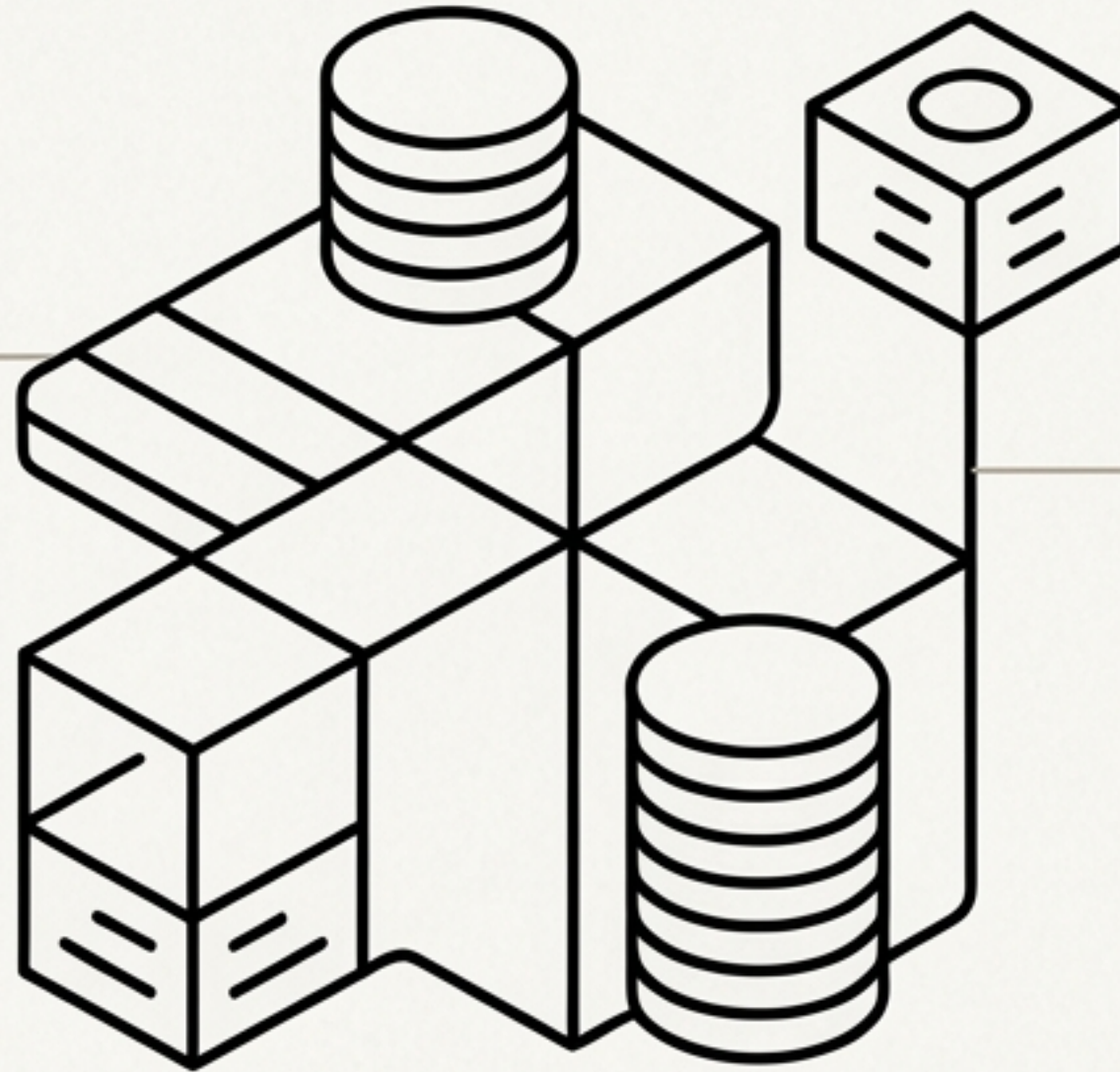
Domine os fundamentos da comunicação em ambientes containerizados e aprenda a construir arquiteturas resilientes usando os drivers de rede do Docker.



Por Que a Comunicação Entre Contêineres é Crucial?

O Desafio Moderno

Aplicações contemporâneas são construídas como microserviços distribuídos: servidores web, bancos de dados, sistemas de cache e APIs precisam se comunicar constantemente.



O Problema Fundamental

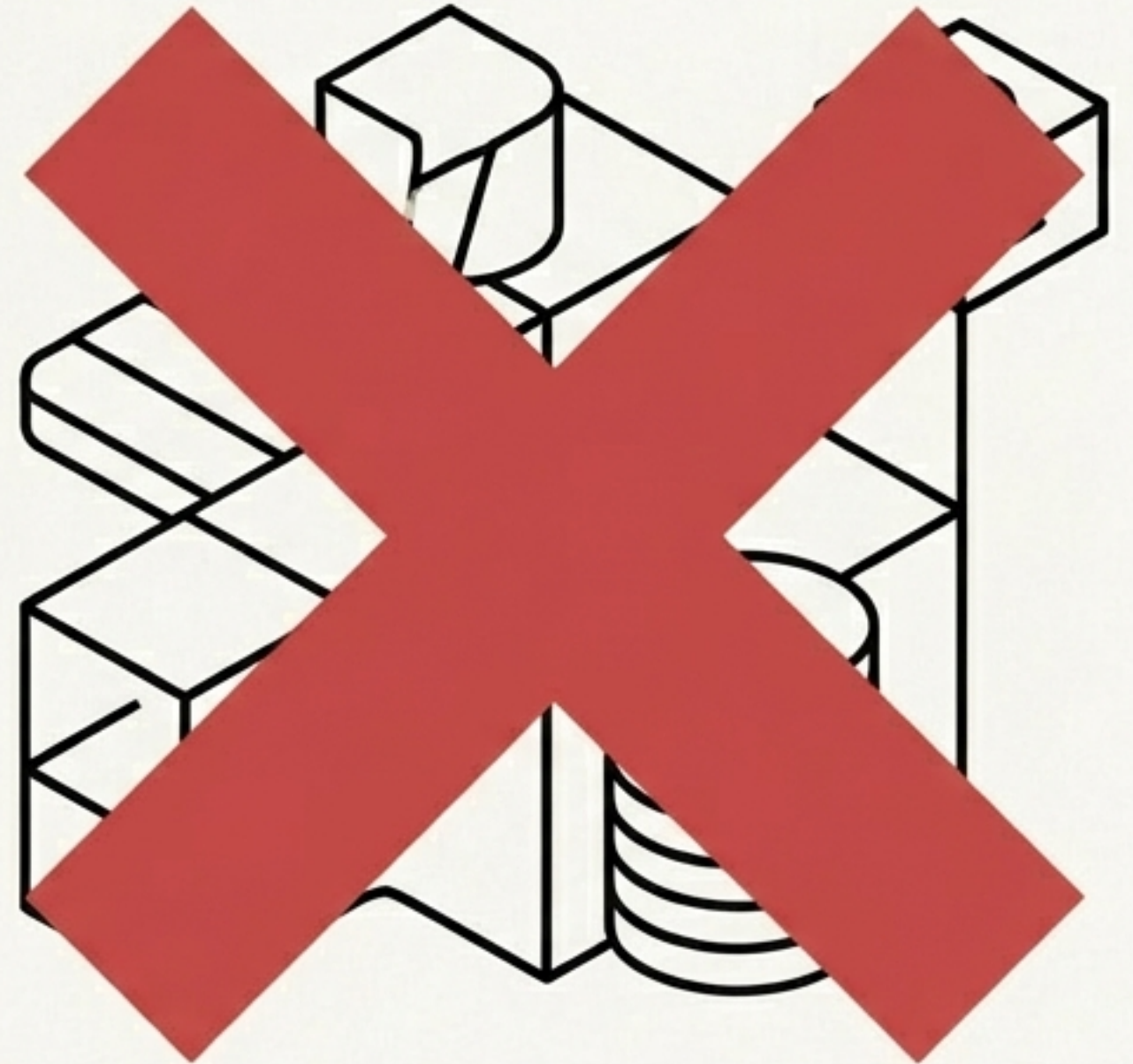
Como o Contêiner A localiza o Contêiner B de forma confiável? Endereços IP são voláteis e mudam a cada reinicialização.

A Tentativa Padrão: A Rede Bridge Automática

Todo contêiner, se não especificado, é conectado à rede `bridge` padrão do Docker. Ela oferece isolamento básico do host.

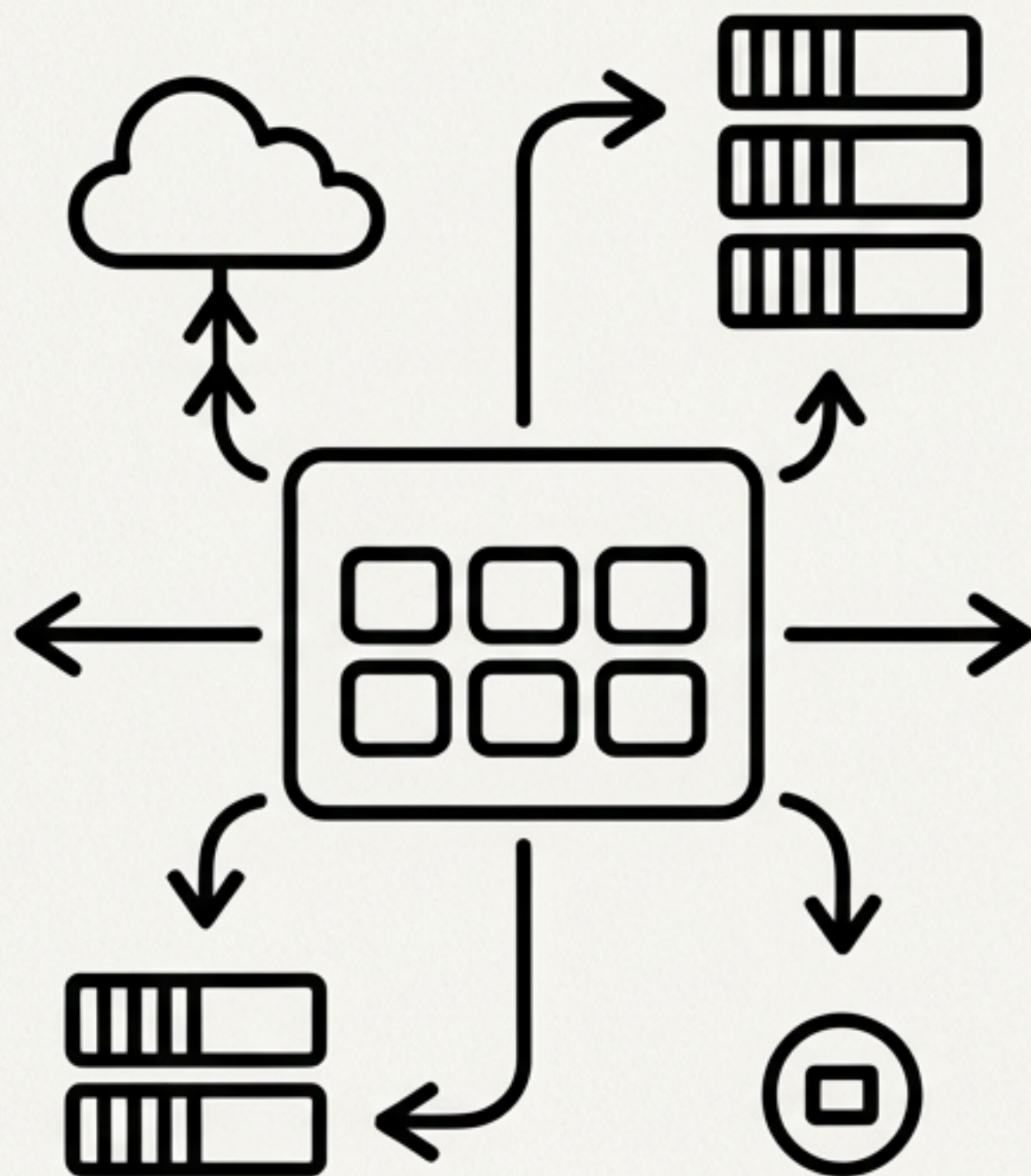
****Limitação Crítica****: A rede bridge padrão **NÃO suporta resolução de nomes (DNS)**. Contêineres só podem se comunicar usando endereços IP, que são voláteis.

```
# A comunicação por nome falha
$ docker exec container_a ping container_b
ping: bad address 'container_b'
```



A Solução Definitiva: A Rede Bridge Customizada

A rede customizada é o coração da comunicação moderna entre contêineres. O Docker cria automaticamente um DNS interno que permite que contêineres se refiram uns aos outros pelo nome do serviço.



01

Criar a Rede

Use `docker network create minha_rede` para criar uma rede isolada.

02

Conectar Contêineres

Lance contêineres com a flag `--network minha_rede` no comando `docker run`.

03

Comunicação por Nome

Contêineres agora se comunicam usando seus nomes. `ping web_app` funciona perfeitamente!

Comparação Direta: Padrão vs. Customizado

Bridge Padrão



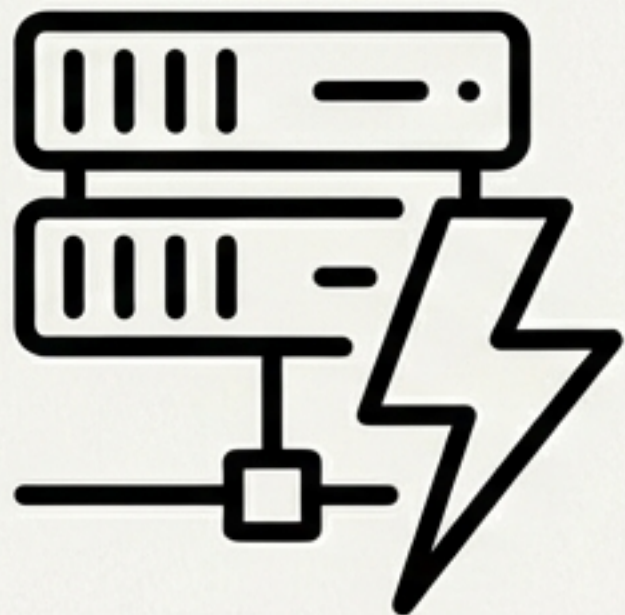
- **Sem DNS interno**
- Comunicação apenas por **IP**
- **IPs mudam** a cada reinício
- **Difícil de manter e escalar**

Bridge Customizado



- **DNS interno automático**
- Comunicação por **nome de contêiner**
- **Nomes são persistentes**
- **Escalável e confiável**

Ferramentas Especiais: Redes Host e None



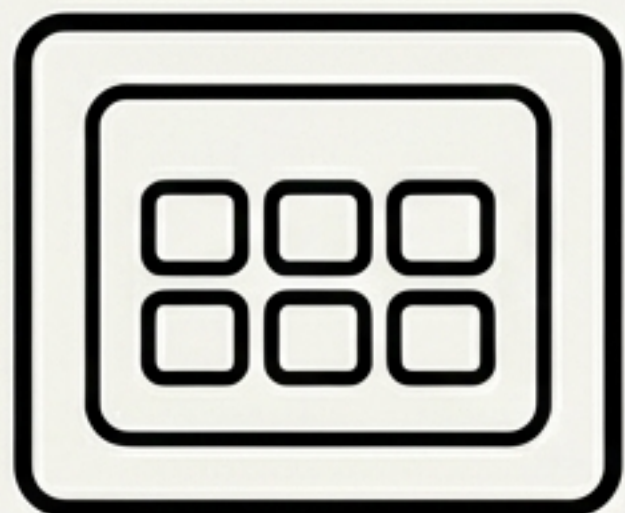
Driver Host

Descrição: O contêiner compartilha o namespace de rede do host, acessando diretamente as interfaces de rede do sistema.

Vantagens: Performance máxima, latência mínima.

Desvantagens: Sem isolamento de portas. Risco de conflitos.

Uso Ideal: Aplicações que precisam de performance de rede extrema.



Driver None

Descrição: Contêiner totalmente isolado, sem qualquer interface de rede. Nenhuma comunicação de entrada ou saída é possível.

Uso Ideal: Processamento em lote, tarefas que não requerem I/O de rede, máxima segurança para processos sensíveis.

Mapeamento de Portas: Controlando o Acesso Externo

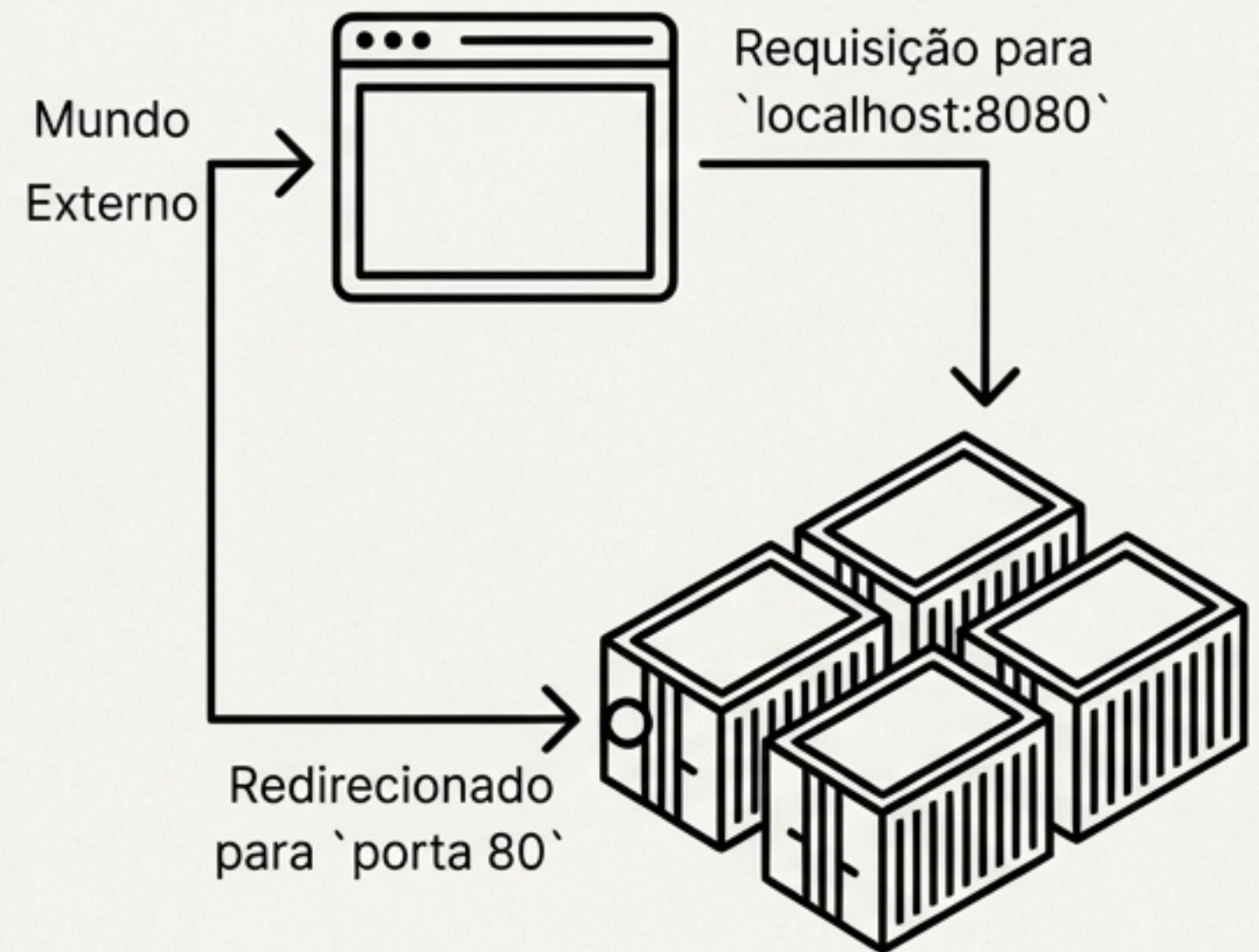
O mapeamento de portas com `-p` controla o acesso do **mundo externo** ao contêiner, **não** a comunicação interna entre contêineres.

Exemplo de Sintaxe

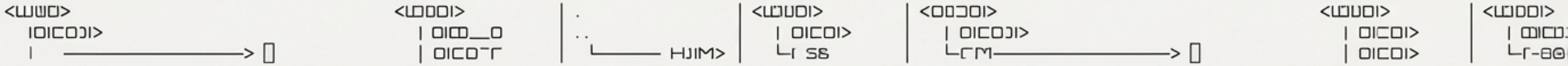
```
# Requisições para localhost:8080 são redirecionadas  
# para a porta 80 dentro do contêiner.  
docker run -p 8080:80 nginx
```

Pontos Importantes

- HOST:CONTAINER: A sintaxe é `-p <Porta_no_HOST>:<Porta_no_CONTÊINER>`.
- Não é necessário para comunicação interna (use o DNS da rede customizada).
- Múltiplos mapeamentos são possíveis.



Gerenciamento de Redes: Comandos Essenciais



docker network create

Descrição: Cria uma nova rede bridge customizada com DNS interno habilitado automaticamente.

Exemplo: `docker network create minha_rede_app`

docker network ls

Descrição: Lista todas as redes disponíveis no sistema.

docker network inspect

Descrição: Exibe detalhes completos de uma rede: subnet, gateway, e contêineres conectados.

Exemplo: `docker network inspect minha_rede_app`

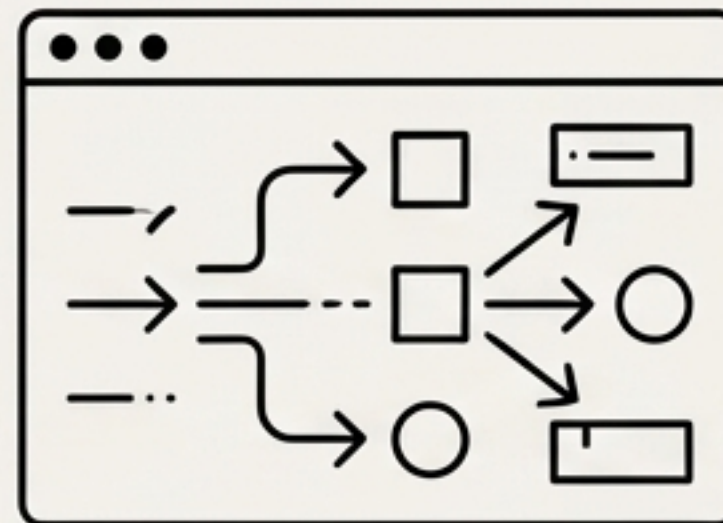
docker network rm

Descrição: Remove uma rede customizada (requer que todos os contêineres sejam desconectados primeiro).

Exemplo: `docker network rm minha_rede_app`

Investigando a Fundo com `docker network inspect`

O comando `docker network inspect` é sua ferramenta de diagnóstico mais poderosa. Ele revela a estrutura completa da rede em formato JSON.



Informações Reveladas

- Subnet: A faixa de IPs disponíveis para os contêineres.
- Gateway: O roteador interno da rede.
- Containers: Lista de todos os contêineres conectados, com seus respectivos IPs.
- Driver: O tipo de rede (e.g., bridge).
- Options: Configurações adicionais.

Exemplo de Saída (simplificado):

```
[
  {
    "Name": "minha_rede",
    "Id": "...",
    "Containers": {
      "container_id_1": {
        "Name": "web_app",
        "IPv4Address": "172.19.0.2/16",
        ...
      }
    }
  }
]
```


Validando a Conectividade: Testes Essenciais

Para provar a conectividade entre contêineres, use ferramentas de **diagnóstico** dentro do próprio contêiner de origem.

1 Acesse o Contêiner

```
docker exec -it <nome_container_origem> sh
```

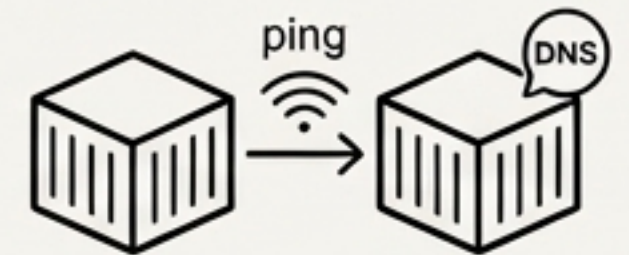
Entra no shell interativo do contêiner.



2 Teste a Resolução de Nome e Rede (Ping)

```
ping <nome_container_destino>
```

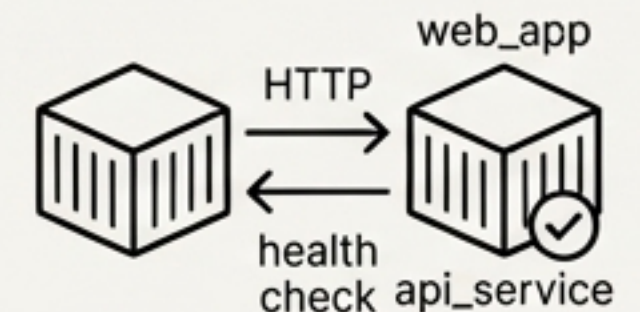
Verifica se o DNS resolve o nome e se há conectividade de rede básica.



3 Teste a Conexão da Aplicação (wget/curl)

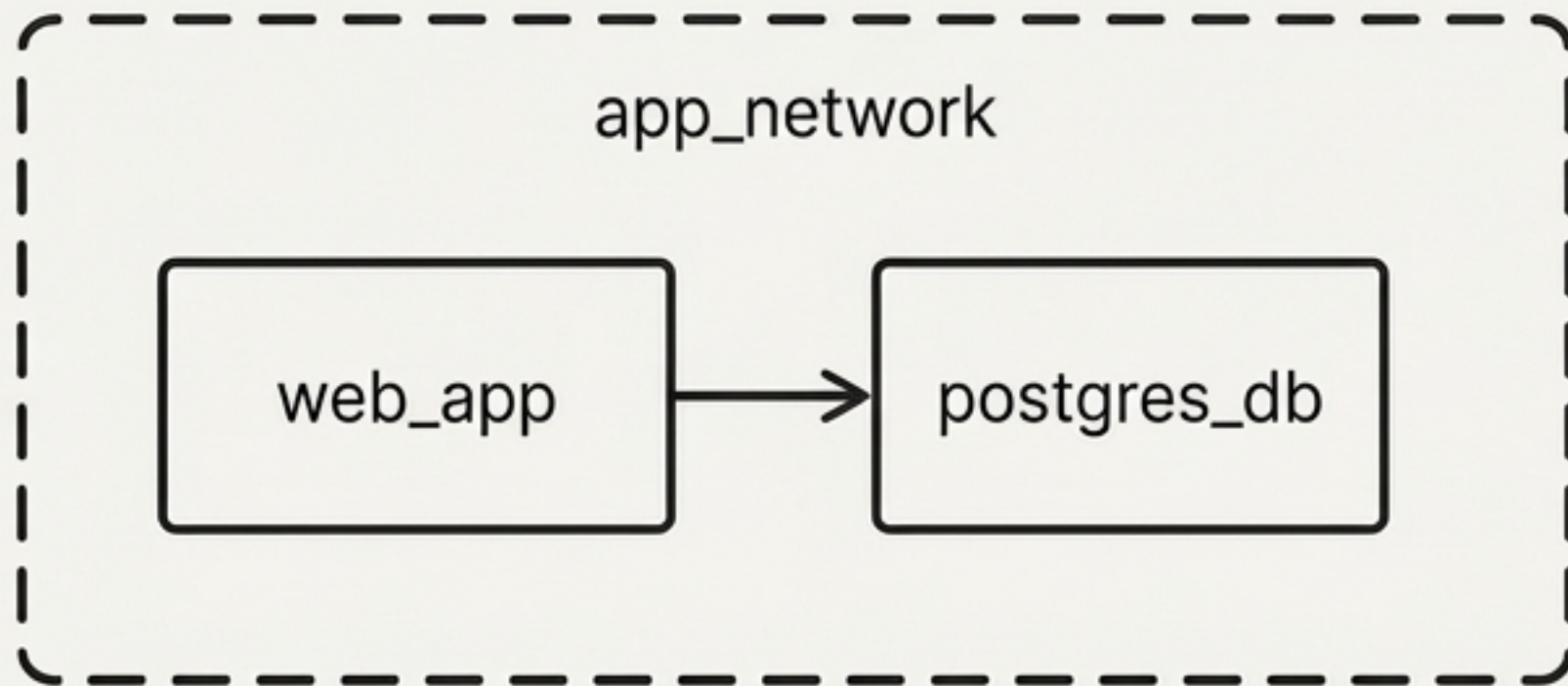
```
wget http://web_app:80 ou  
curl http://api_service:3000/health
```

Simula uma requisição HTTP real à aplicação, usando o nome do serviço.



Exemplo Prático: Aplicação Web + Banco de Dados

Vamos construir uma arquitetura real: uma aplicação Node.js (`web\_app`) conectada a um banco PostgreSQL (`postgres\_db`) usando uma rede customizada.



1. Passo 1: Criar a rede customizada

```
docker network create app_network
```

2. Passo 2: Lançar o banco de dados na rede

```
docker run -d --name postgres_db \
  --network app_network \
  -e POSTGRES_PASSWORD=senha123 \
  postgres:15
```

Passo 3: Lançar a aplicação web na mesma rede

```
docker run -d --name web_app \
  --network app_network \
  -p 3000:3000 \
  -e DB_HOST=postgres_db \
  minha_app:latest
```

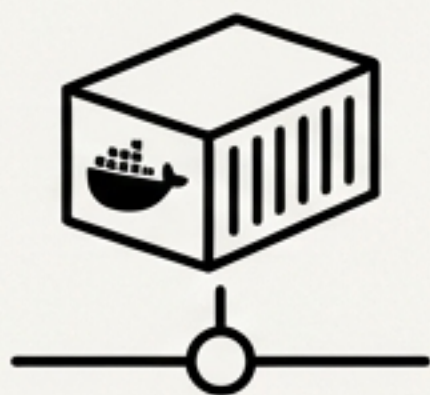
Passo 4: Testar a conectividade

```
docker exec -it web_app ping postgres_db
```


Recapitulando: Pontos-Chave das Redes Docker

Bridge Padrão

- Sem DNS interno, comunicação apenas por IP.
- **Evite para produção.**



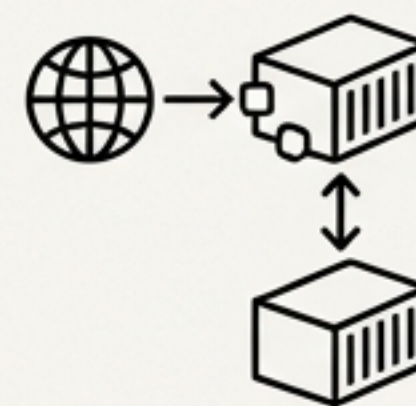
Bridge Customizado

- DNS interno automático, comunicação por nome.
- **Ideal para arquiteturas de microserviços.**



Mapeamento de Portas (-p)

- Controla o acesso **externo** ao contêiner.
- Não afeta a comunicação **interna**.



Lembre-se: Redes customizadas são essenciais para construir arquiteturas escaláveis e confiáveis com Docker.

Próximos Passos: Orquestração com Docker Compose

Agora que você domina redes, o próximo passo natural é o Docker Compose: o gerenciamento de aplicações multi-contêiner usando um único arquivo YAML.

- Aprenda a definir serviços, redes e volumes de forma declarativa.
- Crie ambientes reproduzíveis e fáceis de versionar.
- Simplifique a gestão de arquiteturas complexas como a do exemplo prático.

